Lecture 02


Gestión de Compras
Gestión de Ventas
Gestión de Almacén
odoo
Gestión Financiera
Y mucho más...
Gestión de Personal

Odoo

You can access the course materials via this link

https://maharatech.gov.eg/course/view.php?id=244

Contents

- Odoo Views
- Form View in depth
- Relational fields
- Fields attributes
- onchange decorator
- Buttons and state
- Odoo ORM

Odoo Views

- Odoo Golden Rule: **Every thing is a record**
- Odoo views are database records added to **`ir_ui_view`** table
- As a convention we are adding all views xml related files into ***`views`*** directory
- Odoo base views are ***tree*** and ***form*** views
- Other extra view types are: ***Kanban, Calendar, Gantt, Search***

Odoo Views

Tree View

```
<record id="iti_student_tree_view" model="ir.ui.view">
    <field name="name">Iti Student Tree view</field>
    <field name="model">iti.student</field>
    <field name="arch" type="xml">
        <tree>
            <field name="name"/>
            <field name="age"/>
            <field name="salary"/>
        </tree>
    </field>
</record>
```

Odoo Views

Form View

```
<record id="iti_student_form_view" model="ir.ui.view">
    <field name="name">Iti Student From view</field>
    <field name="model">iti.student</field>
    <field name="arch" type="xml">
        <form>
            <field name="name"/>
            <field name="age"/>
            <field name="salary"/>
        </form>
    </field>
</record>
```

Form View in depth

- **<sheet>** : Shows the form with paper look and feel
- **<group>** : Shows the field label also can be used for splitting fields into columns
- **<notebook>** : Adds tabs to form view
- **<page>** : Adds new tab in the form

Relational fields

1 – Many2one Field

- Added to the many part and relates to the other part
- Has a comodel_name attribute which links to the other model
- A physical field in the database
- Naming convention: single_name_id

```
track_id = fields.Many2one(comodel_name="iti.track")
```

Relational fields

2 – One2many Field

- Added to the other part of the Many2one relation
- Has a comodel_name attribute which links to the other model and inverse_name which relates to the many2one field of the other part
- A logical field (Does not exist in the database)
- Naming convention: single_name_ids
- Must have a many2one field in the other table to link to it

```
student_ids = fields.One2many(comodel_name="iti.student", inverse_name="track_id")
```

Relational fields

3 – Many2many Field

- Added to any of the two parts (can be added to both)
- Has a comodel_name attribute which links to the other model and all other fields are optional
- Creates a new table in the database links both tables
- Naming convention: plural_name_ids

```
skills_ids = fields.Many2many(comodel_name="iti.skill")
```

Relational fields

4 – Related Field

- Relates to a many2one field attribute
- Has the same type of the original field
- Mostly used as readonly field

```
track_id = fields.Many2one(comodel_name="iti.track")
track_name = fields.Char(related="track_id.name")
```

Fields Attributes

ATTRIBUTE_NAME = VALUE

Attribute name	description	values
readonly	Set a field as readonly field	True, False
invisible	Hides a field from a view	True, False
required	Sets a field as a mandatory	True, False

attrs	Set a field as readonly and/or invisible and/or required based on specific domain(condition)	{
		“required”: list of tuples,
		“readonly”: list of tuples,
		“invisible”: list of tuples,
		}

domain	Limit a many2one field options based on a specific condition(s)	List of tuples [(“Field”, “OPERATOR”, VALUE)]

Onchange decorator

- Onchange decorator can be used to **update UI on the fly**
- It can be used to :
  - Change field(s) value
  - Change field(s) domain
  - Popup a warning message to the user

```
@api.onchange('is_accepted')
def _onchange_accepted(self):
    self.salary = 1500
    return {
        'domain': {'track_id': [('is_open', '=', True)]},
        'warning': {'title': "Warning",
                    'message': "The student will be accepted"},
    }
```

Onchange decorator

- When a user changes a field's value in a form (but hasn't saved the form yet), it can be useful to automatically update other fields based on that value.
- The changes performed during the method are then sent to the client program and become visible to the user
- It is possible to suppress the trigger from a specific field by adding on_change="0" in a view

Buttons and states

Inside form view

```
<header>
    <button name="action_approve" string="Approve" type="object"/>
    <field name="state" widget="statusbar"
           statusbar_visible="draft,approved"/>
</header>
```

Inside model class

```
@api.multi
def action_approve(self):
    for rec in self:
        rec.state = "approve"

#self.write({"state": "approve"})
```

Odoo Orm

- **Create:**
  - Takes a dictionary of field values, or a list of such dictionaries, and returns a recordset containing the records created

```
>>> self.create({'name': "Joe"})
res.partner(78)
>>> self.create([{'name': "Jack"}, {'name': "William"}, {'name': "Averell"}])
res.partner(79, 80, 81)
```

Odoo Orm

- **Write:**
  - Takes a number of field values, writes them to all the records in its recordset. Does not return anything

```
self.write({'name': "Newer Name"})
```

Odoo Orm

- **Search:**
  - Takes a search domain, returns a recordset of matching records.
  - Can return a subset of matching records (offset and limit parameters) and be ordered (order parameter)
  - to just check if any record matches a domain, or count the number of records which do, use search_count()

```
>>> # searches the current model
>>> self.search([('is_company', '=', True), ('customer', '=', True)])
res.partner(7, 18, 12, 14, 17, 19, 8, 31, 26, 16, 13, 20, 30, 22, 29, 15, 23, 28, 74)
>>> self.search([('is_company', '=', True)], limit=1).name
'Agrolait'
```

Odoo Orm

- **Browse:**
  - Takes a database id or a list of ids and returns a recordset, useful when record ids are obtained from outside Odoo (e.g. round-trip through external system)

```
>>> self.browse([7, 18, 12])
res.partner(7, 18, 12)
```

Odoo Orm

- **unlink:**
  - Deletes the records of the current set

```
self.unlink()
```